

Loan-Approval-Prediction — Assignment Presentation

This deck summarizes a machine learning pipeline for predicting loan approvals: problem statement, data, feature engineering, modeling, evaluation, visual results, and next steps. Code, plots, and PDFs with images were provided and are incorporated where relevant.

Mohit Kumar

<https://github.com/MohitMintorian>

Kumarmohitamu@gmail.com



Motivation & Objectives

Problem

Manual loan decisions are slow, inconsistent, and subject to bias.

Goal

Build an automated model to predict Approve/Reject with high accuracy and fairness.

Impact

Faster decisions, consistent criteria, scalable processing, improved applicant experience.



Dataset & Resources

Source: Project dataset (CSV). Link:

drive.google.com/loan_dataset

- Key fields: income_annum, loan_amount, cibil_score, loan_term
- Assets: residential, commercial, luxury, bank_asset_value
- Demographics: no_of_dependents, education, self_employed



LOAN_AMOUNT	INCOME	CIBIL_SCORE	LOAN_TERM	APPROVED

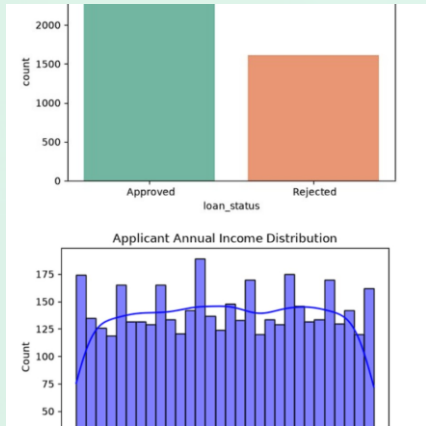
Data Preparation & Feature Engineering



Steps applied:

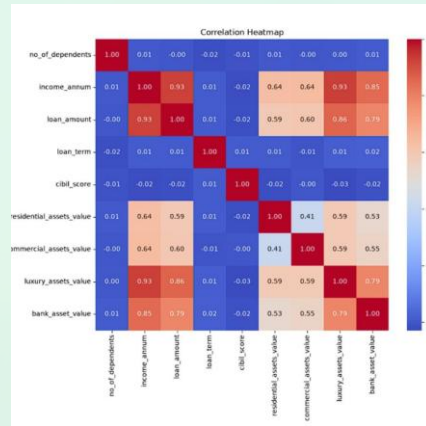
- Dropped identifier column (loan_id); normalized column names (strip, lower)
- Checked missing values — none reported for key fields in provided run
- Encoded categoricals with LabelEncoder (education, self_employed, loan_status)
- Engineered features: **Debt to income ratio** = $\text{loan_amount} / \text{income_annum}$ and **total_assets** (sum of asset fields)

Exploratory Data Analysis — Key Visuals



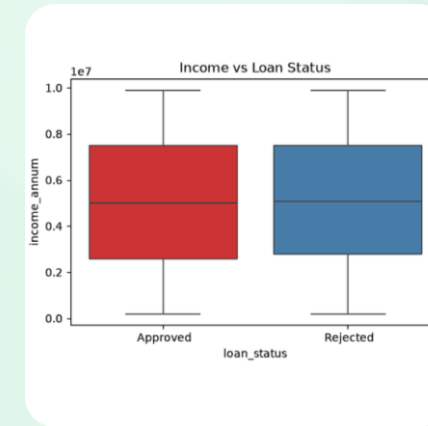
Loan Status & Income

Distribution shows more Approved than Rejected; income histograms reveal broad spread across applicants.



Loan Amount & CIBIL

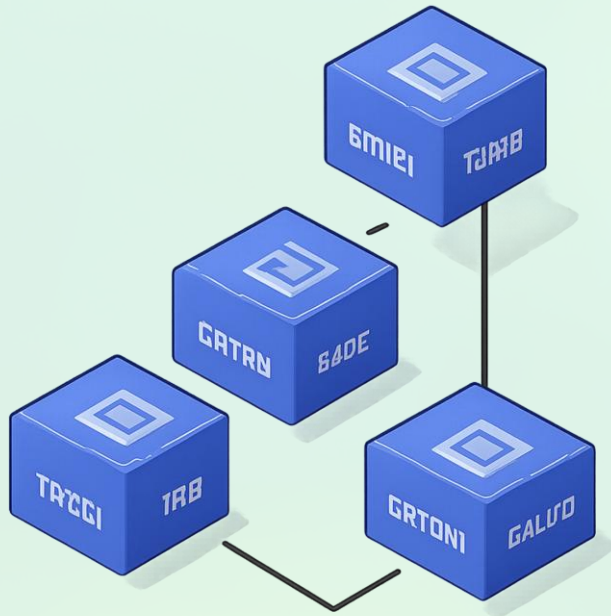
Loan amounts are right-skewed; CIBIL scores are roughly evenly distributed with peaks around mid-range scores.



Correlation Heatmap

Strong positive correlations between income, loan_amount, and several asset values — useful for feature interactions.

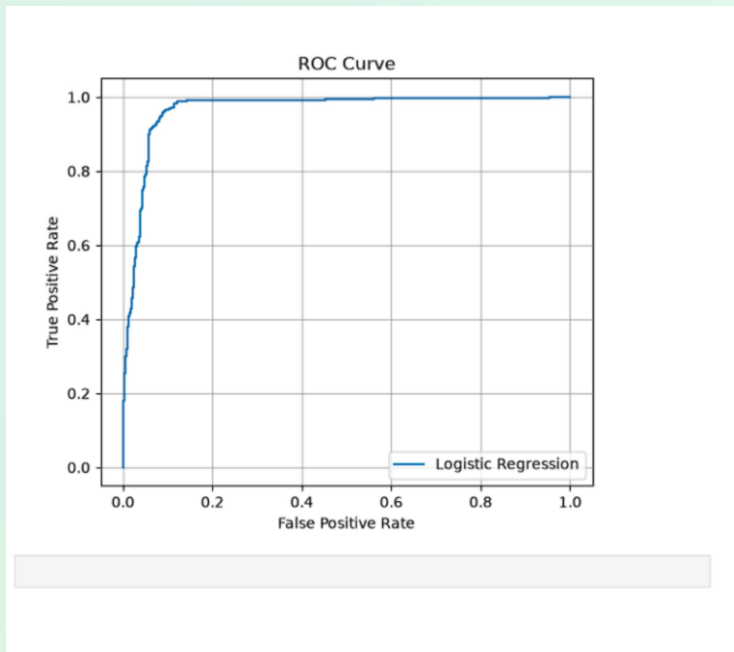
Modeling Approach



Implementation details (from provided code):

- Model: Logistic Regression (max_iter=1000, class_weight="balanced")
- Split: train_test_split (test_size=0.2, random_state=18, stratify=y)
- Scaling: StandardScaler on numeric features
- Features used: 13 engineered + original numeric fields (listed in code)

Evaluation Metrics

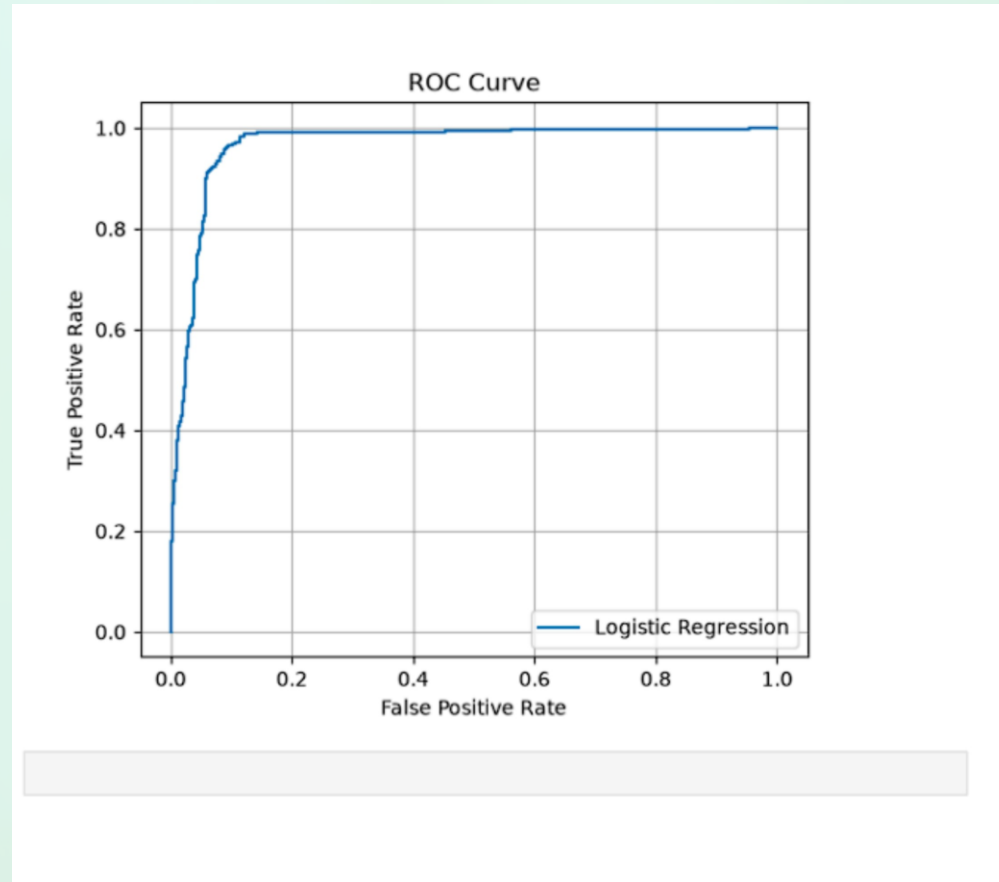
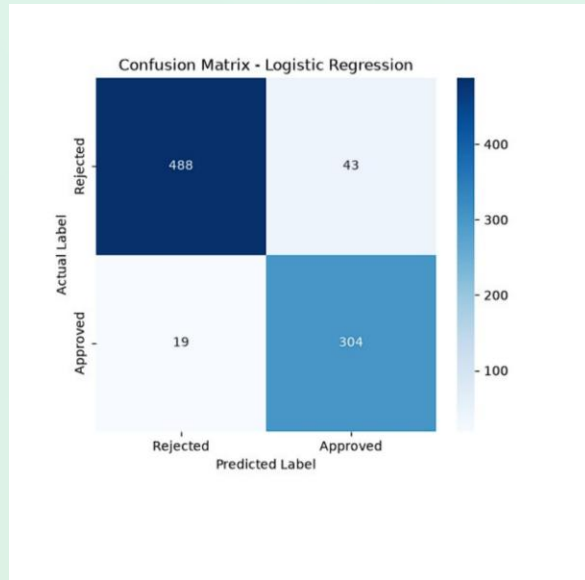


Reported performance on test set:

- Accuracy: 0.9274
- Precision: 0.8761
- Recall: 0.9412
- F1 Score: 0.9075
- ROC-AUC: 0.9660

Classification report indicates strong class performance with slightly lower precision for the positive class.

Confusion Matrix & ROC



Interpretation: High true positive and true negative counts (304 TP, 488 TN). ROC curve demonstrates strong separability — good discrimination between classes.

Limitations, Bias & Robustness

Potential Bias

LabelEncoder may encode categories without addressing protected attributes. Need fairness checks to avoid demographic bias.

Data Quality

Current run shows no missing values — verify across raw uploads. Check for outliers and data entry errors in asset fields.

Model Generalization

Logistic regression performs well here; consider tree-based models and cross-validation for comparison and robustness.

Conclusions & Next Steps



Summary:

- The logistic regression pipeline achieved strong performance (ROC-AUC ≈ 0.966).
- Feature engineering (debt-to-income, total_assets) added predictive value.

Recommended next steps:

1. Run k-fold cross-validation and compare additional algorithms (Random Forest, XGBoost).
2. Perform fairness audits and mitigate bias (reweighing, adversarial debiasing).
3. Build a lightweight API for real-time scoring and add model monitoring.

❏ **Thank you for reviewing this work.**